Easy Trip

Project Handoff Document

For Claude Cowork — Full context for zero-prior-knowledge onboarding

1. Project Goal

What we're building:

Easy Trip is an AI-powered US trip planning web app. Users fill out a multi-step form with their destination, dates, budget, travel style, and preferences. The app uses the Claude API to generate a complete personalized day-by-day itinerary including hotels, flights, restaurants, routes, and attractions.

Why:

Trip planning is time-consuming and overwhelming. Easy Trip removes that friction — users answer a few questions and get a complete trip plan in minutes instead of hours of research.

Target market: US travelers first. International expansion planned later.

Business model: Free to use initially. Stripe payments planned for a premium tier later.

2. Key Decisions Made

Next.js 16 with TypeScript

Handles both frontend and backend in one framework. Vercel (deployment) is built by the same team so integration is seamless.

Tailwind CSS v4

Used for all styling. No external UI libraries (no shadcn, no MUI). Everything is custom built.

Supabase for database and auth

Chosen for its generous free tier, built-in authentication, and simple JavaScript SDK. NOT YET INTEGRATED — planned for Phase 3.

Vercel Blob for video storage

Background videos are too large for GitHub. Hosted on Vercel Blob and referenced by URL. Videos are in .gitignore and should NEVER be pushed to GitHub.

Google Places API via backend route

City/destination autocomplete calls Google Places through a Next.js API route (app/api/places/route.ts) instead of directly from the browser. Browser CORS blocks direct calls, and keeping the API key server-side is more secure.

localStorage for form persistence

User form progress saved to localStorage key 'easytrip-plan' so refreshing restores progress. Temporary storage only — Supabase handles permanent trip saving after login is built.

ssr: false for the plan page

The /plan page had persistent hydration errors when server-rendered. Fixed permanently using next/dynamic with ssr:false. Page split into page.tsx (wrapper) and PlanPageClient.tsx (actual form).

Built-in city dataset → Google Places

Started with hardcoded city list but switched to Google Places Autocomplete because it covers national parks, beaches, and smaller destinations the dataset missed.

3. Current State

Completed ■

Homepage (app/page.tsx):

- Navbar with Easy Trip logo (SVG pin + clock icon)
- Hero section with cycling video background (8 videos on Vercel Blob)
- How It Works section (3 steps, background image at /public/howitworks-bg.jpg)
- Features section (6 cards with custom background images at /public/features/card1-6.jpg)
- CTA section and Footer
- All buttons wired up — scroll links work, /plan buttons route correctly
- Horizontal scroll overflow fixed, video lag fixed (compressed to ~41MB total)

Trip Planning Form (app/plan/PlanPageClient.tsx):

Step 1 — Destination & Dates:

- State dropdown (all 50 US states, filters as user types)
- City/area input using Google Places Autocomplete (cities and national parks only, no streets)
- Multiple destinations supported (up to 5 stops), numbered, with route preview
- Add/remove/reorder destinations
- Start date minimum = today with validation, end date must be after start date
- Shows 'Your trip is X days long' confirmation
- Next button disabled until all fields valid

Step 2 — Customize Your Experience:

- Travel style multi-select (Adventure, City & Culture, Food & Dining, Relaxation, Family Friendly, Nightlife)
- Transportation: Flying / Driving / Either/Flexible
- Accommodations: Hotel / Airbnb/VRBO / Hostel / Camping / Flexible

Step 3 — Budget & Group Size:

- Group size: Just Me / Couple / Small Group / Large Group
- Small Group → number input (3-5), Large Group → number input (6+)
- Total budget input with $ prefix, comma formatting, synced slider ($100 - $10,000+)
- Shows per person budget live (total / people)
- Flight budget per person (required if Flying, optional if Either/Flexible, hidden if Driving)
- Accommodation budget (required, varies by type chosen in Step 2, hidden if Camping)
- Real-time budget validation — each input cannot exceed remaining budget per person
- Shows remaining budget after each expense
- Final budget summary card with color coding (green/yellow/red based on remaining buffer)
- localStorage persistence — refreshing restores all data and returns to correct step
- Restart button (red, top right, hover tooltip warning, confirmation dialog)

Step 4 — Must Haves & Submit:

- Optional text area for specific requests
- 'Plan My Trip' submit button with loading state

Form features across all steps:

- Progress bar at top (4 steps)
- Summary sidebar on desktop showing trip details so far
- Smooth transitions between steps, mobile responsive
- localStorage auto-save on every field change
- Restart button visible on every step

In Progress ■

- Google Places Autocomplete city search — works but needs improvement. Short queries (2-3 chars) sometimes return irrelevant results. Bounding box approach partially implemented.

Not Started ■

- AI itinerary generation (Claude API integration)
- Results/itinerary display page (app/results/page.tsx)
- Google Maps integration (routes, drive times)
- Amadeus API (live flight data)
- Google Places for restaurants
- RapidAPI (hotels, gas prices)

- NPS.gov API (national parks data)
- Supabase auth (user accounts)
- Saving trips to Supabase
- Stripe payments
- App renaming (currently 'Easy Trip', developer wants something more unique)
- Custom domain purchase

4. Technical Context

Stack

Framework	Next.js 16.2.1 (App Router, Turbopack)
Language	TypeScript
Styling	Tailwind CSS v4 (no external UI libraries)
Database	Supabase (not yet integrated)
AI	Anthropic Claude API (@anthropic-ai/sdk v0.80.0)
Deployment	Vercel (Hobby plan, free)
Node version	v22.20.0
npm version	10.9.3

File Structure

```
/Users/justin.chiang/Documents/Easy-Trip/
■■■ app/
■   ■■■ page.tsx                    ← Homepage
■   ■■■ layout.tsx                  ← Root layout
■   ■■■ globals.css                 ← Global styles
■   ■■■ api/
■   ■   ■■■ places/
■   ■       ■■■ route.ts            ← Google Places API backend route
■   ■■■ plan/
■       ■■■ page.tsx                ← Thin wrapper (ssr:false dynamic import)
■       ■■■ PlanPageClient.tsx      ← Full multi-step form (use client)
■■■ public/
■   ■■■ features/
■   ■   ■■■ card1.jpg through card6.jpg  ← Feature section backgrounds
■   ■■■ howitworks-bg.jpg           ← How It Works section bg
■■■ .env.local                      ← API keys (NEVER push to GitHub)
■■■ .gitignore                      ← Includes public/videos/
■■■ next.config.ts
■■■ package.json
■■■ tsconfig.json
```

Environment Variables (.env.local)

Variable	Purpose
`GOOGLE_API_KEY`	Google Maps, Places, Directions (server-side only)
`RAPIDAPI_KEY`	Hotels, flights, gas prices
`NPS_API_KEY`	National Parks Service data
`ANTHROPIC_API_KEY`	Claude AI itinerary generation
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase public anon key

Google APIs Enabled

- Maps JavaScript API
- Places API
- Places API (New)
- Directions API
- Geocoding API
- Distance Matrix API

Vercel Setup

Project name	`easy-trip`
Plan	`Hobby (free)`
Live URL	`easy-trip-steel.vercel.app`
GitHub repo	`github.com/JustinC333/Easy-Trip`
Blob store	`easy-trip-videos (SFO1, Public)`
Blob base URL	`https://ucur14dm16l5fxk8.public.blob.vercel-storage.com/`
Videos	`bg1.mp4 through bg8.mp4 on Blob (NOT in GitHub)`

Dependencies (package.json)

```
dependencies:
  @anthropic-ai/sdk: ^0.80.0
  @supabase/supabase-js: ^2.99.3
  next: 16.2.1
  react: 19.2.4
  react-dom: 19.2.4

devDependencies:
  @tailwindcss/postcss: ^4.2.2
  tailwindcss: ^4
```

```
typescript: ^5
eslint: ^9
eslint-config-next: 16.2.1
```

5. Known Issues / Blockers

■■ Google Places Autocomplete — partial fix

The city search works but short queries (2-3 characters) sometimes return irrelevant results. Current approach uses bounding box coordinates per state. Works well for longer queries but needs refinement for short ones. API route is at app/api/places/route.ts. Uses types:'(cities)' for cities and a separate call for national parks, filtered and combined. Session tokens used to minimize billing.

■■ Hydration errors — fixed but monitor

The /plan page had persistent React hydration errors from server/client render mismatches in the ProgressBar component and localStorage access. Fixed by splitting into page.tsx (wrapper with ssr:false) and PlanPageClient.tsx (use client). If new hydration errors appear, the cause is almost always inline styles with dynamic values — convert to Tailwind classes to fix.

■■ Google API key restrictions

The Google API key had HTTP referer restrictions that blocked server-side calls. These were removed. The key currently has NO restrictions. Consider adding IP-based restrictions when deploying to production.

■■ App name undecided

The app is currently called 'Easy Trip' but the developer wants to rename it before launch. Name is undecided. All instances of 'Easy Trip' are in UI files and can be replaced with a single Claude Code prompt when ready.

6. Next Steps

Immediate — Fix City Search (Step 1)

The Google Places search needs one more refinement pass. When user types 2-3 characters, results should immediately show relevant cities and national parks in the selected state.

Test cases that MUST work:

- "sa" in California → San Diego, Sacramento, Santa Barbara
- "yo" in California → Yosemite National Park
- "las" in Nevada → Las Vegas
- "mia" in Florida → Miami

Phase 1 — Build AI Itinerary Generation

Create app/api/generate/route.ts — a Next.js API route that receives the complete form data, builds a detailed prompt for Claude API, calls @anthropic-ai/sdk, and returns a structured JSON itinerary.

The prompt to Claude must include:

- All destinations and stops in order
- Travel dates and number of days
- Total budget, per person budget, flight budget, accommodation budget
- Group size and composition
- Travel style preferences (adventure, food, etc.)
- Transportation method and accommodation type
- Any must-haves from Step 4

Phase 1 — Build Results Page

Create app/results/page.tsx to display the generated itinerary in a clean, day-by-day format with hotel cards, restaurant suggestions, drive times, and activity recommendations.

Phase 2 — Connect Live APIs One by One

1	**Google Maps**	Drive times and routes between stops
2	**Google Places**	Restaurant recommendations per city
3	**Amadeus**	Live flight options and prices
4	**RapidAPI**	Hotel prices and availability
5	**NPS.gov**	National park details, hours, fees

Phase 3 — Supabase User Accounts

- User signup/login with Supabase Auth
- Save generated trips to Supabase database
- View and manage past trips
- Share trips via unique URL

Phase 4 — Polish & Launch

- Finalize app name and update all instances in code
- Purchase domain and configure on Vercel
- Mobile responsiveness audit
- Performance optimization
- Add Stripe for premium tier